Tournament Management Platform

Master Plan, Part 1 of 4: Scope, Architecture, Repository, Risks and First Task

Document set. (1) This file. (2) Sequential Roadmap, Phase A to deployment. (3) Database Design. (4) Master Checklist, accounts, milestones, pitfalls. Nothing here starts implementation; per your instructions I wait for your review.

Input note. I received two attachments: your master prompt and the Website Blueprint. The Blueprint (section 17) is the only database planning source, so no separate database PDF was available. I treated the Blueprint as the source of truth. Anything I add is labelled **[REC]** (recommendation) or **[ASSUMPTION]**.

1. Understanding of the project scope

A public, no-login tournament site (browse tournaments, fixtures, results, standings, archives, watch live auctions) plus authenticated roles: **Player**, **Organizer** (admin-approved), and exactly one **Global Administrator**. Matches are played externally; the site tracks status and publishes results only after organizer confirmation.

1.1 Feature groups extracted from the Blueprint

Group	Established requirements (Blueprint section)
Identity and access	Email+password, no email verification (3); roles (2); organizer applications and approval (3); single admin bootstrapped securely; suspensions; admin-assisted recovery; deletion requests; rate limits (3, 20)
Tournament setup	Three formats; public/invite-only registration; auto/manual approval; deadlines, reopening; participant cap; waiting list; rule edits any time with impact preview, recalculation, history; lifecycle statuses validated by backend (4)
Registration	Eligibility, deadline, duplicate and capacity checks under concurrency; withdrawal windows; organizer exceptions; notifications (5)
Teams	Captain interest and selection (organizer final); separate auction-eligibility decision; live auction OR manual assignment; squad limits enforced for both (6)
Live auction	Public read-only room; captains bid; server validates X+Y increments; organizer Done -> server countdown; one bid per captain per round; atomic simultaneous bids; append-only undo; purse/squad checks; unsold rounds; finalization (7)
Fixtures and standings	Auto/manual fixtures; knockout bracket progression; round-robin; groups -> knockout; configurable points, tie-breakers, qualification; audited overrides; confirmed results only (8, 9)
Results	Manual entry and CSV import with preview/validation; organizer confirmation; corrections preserving original; recalculation; supported stats only; live status without scores (10)
Communication	Persistent in-site notifications; announcements with pin; Socket.IO live updates; configurable polling fallback (11)
Pages and dashboards	Homepage, discovery/filters, tournament detail tabs, player/organizer/admin dashboards (12)
Privacy and governance	History visibility control; anonymize on deletion without destroying results; suspension/removal; cancellation/rescheduling with reasons; audit logs; admin interventions audited (13 to 16)
Output and ops	CSV/PDF exports with formula-injection protection; low-cost hosting; backups with tested restore (21, 22)

1.2 Contradictions, ambiguities and missing decisions

*Each item has my recommended resolution. Items marked **BLOCKING** must be decided before the named phase.*

#	Issue	Recommended resolution	Needed by
1	Rules editable "at any time" (4) versus auction integrity (7). Changing purse, increment or squad size mid-auction would corrupt state.	[REC] Auction parameters freeze once auction status is "live"; they can only change through the admin-only exceptional correction path (audited). All other rules stay editable with impact preview.	Phase I
2	"Bidding round" is undefined. Is a round the interval between two Done clicks? The rule "a captain cannot bid again in the same round" depends on it.	[REC] Round = the bidding window between two organizer Done clicks (the first window opens with the lot; a bid accepted during a countdown cancels it and opens a new window). Each captain may have at most one accepted bid per window; the roundNumber is stored on every bid. Confirm this matches your intent. **BLOCKING** for Phase M.	Phase L
3	Flow gap: after Done starts the countdown, an accepted bid cancels it, but the blueprint needs the organizer to press Done again.	[REC] Keep as written: a bid during countdown stops the timer and opens a new round; organizer presses Done to restart. Optional organizer setting "auto-restart countdown" can come later.	Phase M

#	Issue	Recommended resolution	Needed by
4	Invite-only mechanism is not specified (invite links, codes, email list?).	[ASSUMPTION] Organizer generates single-tournament invite codes/links (optionally bound to one user). Needs an **invites** collection. Essential for first release because invite-only is a stated feature.	Phase J
5	Sport/game is never named. Standings tie-breakers and the CSV scorecard are sport-specific; the term "scorecard" suggests cricket.	[ASSUMPTION] Keep engine sport-neutral (win/draw/loss, points, generic tie-breakers). Sport-specific stats live in the isolated CSV adapter. **BLOCKING**: confirm sport and provide a sample CSV before Phase S.	Phase P
6	Two administrator-assisted flows (recovery) require a token but the blueprint stores none.	[REC] Token hash and expiry live on accountRequests (not users). Single-use, 30-60 min expiry, revokes all sessions on use.	Phase G
7	Section 17.5 stores squad on team AND teamAssignments exists (17.6).	[REC] teamAssignments is the source of truth; teams keeps only counters (squadCount, purseRemaining) updated in the same transaction.	Phase F
8	standingsSnapshots is "optional" yet corrections must preserve calculation history (9).	[REC] Standings are computed from confirmed results (pure function) and cached; a snapshot is written on every confirm/correct/override/rule change. Essential for first release because the blueprint demands history.	Phase P
9	Withdrawal windows (5) do not say what happens to a withdrawing captain or sold player.	[REC] Pre-auction: withdrawal also clears captain status. After auction/team assignment: organizer-only exception that triggers the removal workflow (14).	Phase J
10	"Financial or administrative accountability" in 3 implies money, but purses are virtual.	[ASSUMPTION] No real money or payments. Purse values are virtual credits. Interpreted as accountability for auction records only.	Phase C
11	Live match "status only, no scores" versus fixtures list showing results.	[REC] Public API exposes score fields only when fixture has a confirmed result. Unconfirmed/imported data is stored but never returned by public endpoints.	Phase R
12	Organizer applications can be re-submitted after rejection? Not stated.	[REC] Allow one pending application at a time; rejected users may re-apply after viewing the reason.	Phase G
13	Session storage: blueprint offers cookies OR tokens.	[REC] Server-side sessions in MongoDB with an opaque httpOnly cookie (details in section 2). Required for real revocation after recovery and suspension.	Phase D

2. Technology stack and architecture

2.1 Final stack decisions

Concern	Decision	Why / notes
Language	**TypeScript** in both client and server	Large domain model, many state machines (tournament, registration, lot, fixture). Compile-time safety pays off. Share Zod schemas where practical. [REC] Not a change to your stack, only an addition.
Repository	**One monorepo**: client/, server/, docs/	Blueprint decision. Separate installs per app, root convenience scripts only. No npm workspaces needed at first; keep it simple on Windows.
Frontend	React 18/19 + Vite + Tailwind CSS + React Router (data router)	Use the Tailwind setup matching the installed major version (v4 uses the Vite plugin and CSS import; v3 uses tailwind.config + PostCSS). Verify at Phase E; do not mix.
Server data fetching (client)	TanStack Query	[REC] Caching, polling fallback, invalidation after Socket.IO events. Small addition, high value.
Backend	Node.js LTS + Express + Mongoose	Run on current Node LTS. ESM modules with tsx for dev, tsc build for prod.
Validation	Zod	Request validation middleware + shared types. Also validates parsed CSV rows.
Database	MongoDB Atlas (free shared tier for dev and early prod, separate clusters/DB users)	Atlas free tier is a replica set, so multi-document transactions work. Verify current limits at Phase C.
Real-time	Socket.IO on the same Node process	Single instance, no Redis adapter initially. Namespaces: /auction, /live. Rooms per tournament.
Auth	Server-side sessions, httpOnly Secure SameSite cookie, Argon2id (argon2 package) or bcrypt	Argon2id needs a native build; if Windows build tooling causes trouble, use bcrypt (cost >= 12) via bcryptjs or argon2 prebuilt binaries.
CSV	csv-parse (or PapaParse) behind an adapter interface	Parser stays a swappable adapter until a real sample exists.
PDF	PDFKit (or pdfmake) on demand	Avoid headless-browser PDF (memory-heavy on free hosts).
Logging	pino + pino-http	Structured logs; redact cookies, passwords.
Security middleware	helmet, cors (allow-list), express-rate-limit, a CSRF defence	CSRF: SameSite=Lax plus required custom header/Origin check on state-changing requests.
Testing	Vitest (unit), Supertest + mongodb-memory-server **replica-set mode** (API/integration), React Testing Library, Playwright (E2E), socket.io-client (socket tests)	Replica-set test DB is mandatory because transactions need it.
Quality tooling	ESLint, Prettier, Husky + lint-staged (optional)	Introduced in Phase B so style is consistent from commit one.
CI/CD	GitHub Actions	Lint, typecheck, test on every PR; deploy on merge to main.
Hosting (candidates, verify)	Frontend: Cloudflare Pages / Netlify / Vercel. Backend: Render / Railway / Fly.io / a small VPS. DB: Atlas.	Pick after re-checking limits (cold starts, WebSocket support, sleep policies, free-tier changes). Prefer a host that keeps WebSockets alive. See deployment phase.

2.2 Architecture overview (modular monolith)

One Express app, one process, one MongoDB database. Code is separated by **domain modules**; modules talk through service functions, never by reaching into each other's models freely.

- **Layering inside each module:** routes (HTTP only) -> controller (parse/validate, call service) -> service (business rules, transactions) -> repository/model (Mongoose). Business rules never live in routes or React components.
- **State machines in code:** tournament status, registration status, auction lot status, fixture status each have an explicit transition table enforced in the service layer. Invalid transitions return a typed error.
- **Authorization:** middleware chain authenticate -> requireRole -> loadTournament -> requireOwnerOrAdmin -> requireTournamentState. The admin override path is explicit and always audited.

- **REST + Socket.IO:** every mutation is a REST call (bids included) so validation, rate limits and audit logging apply uniformly. Sockets are used mainly to broadcast state to clients. Bids go through REST to a single service that performs the atomic update, then the server emits the new state. [REC] This avoids a second, divergent write path.
- **Auction concurrency:** single-process, so use (a) a conditional atomic update on the lot document (filter on status, version and current price) inside a transaction that also updates the bid log, and (b) a per-lot in-memory mutex as defense in depth. Server persists countdown *endsAt* (absolute timestamp) so restarts can resume; clients render countdown from server time offset, never from their own clocks.
- **Derived data:** standings and qualification are pure functions of confirmed results plus current rules, recomputed in a service and cached with snapshots. Never hand-edited except via audited overrides.
- **File processing:** CSV uploaded to memory (multer memoryStorage, small size cap), parsed, validated, stored as an importJob with status "preview" and never touches results until confirmed.
- **Reports:** generated on demand and streamed; no permanent file storage in version 1.
- **Background work:** only light timers inside the Node process (registration close, auction countdown, expiry of reset tokens). All timers are re-derived from database timestamps at boot, and critical transitions are also checked lazily on access so a sleeping/restarted host cannot skip them.
- **Time:** store UTC; display in the user's local zone; organizers' deadlines are entered with an explicit timezone.

2.3 Environments

Env	Database	Where it runs	Secrets
Development	Atlas dev DB (own DB user) or local mongod replica set	Your Windows 11 laptop	.env files (git-ignored)
Test	In-memory MongoDB replica set per test run	Laptop and GitHub Actions	Fixed fake values in test setup
Production	Separate Atlas cluster/project, separate DB user, IP/network rules	Hosting provider	Provider secret manager

3. Repository and folder structure

Single repo named **tournament-platform**. TypeScript throughout. Differences from your example tree are marked.

```
tournament-platform/
  .github/workflows/        CI (lint, typecheck, test) and deploy       [added]
  client/
    src/
      app/                  router, providers, query client
      components/{common,tournament,fixtures,auction}/
      pages/{public,auth,player,organizer,admin}/
      features/{tournaments,registrations,auction,results,notifications}/
      services/             API client, socket client
      hooks/  types/  utils/
    index.html  vite.config.ts  tsconfig.json  .env.example
  server/
    src/
      config/               env validation (Zod), db connection
      middleware/           auth, roles, validate, errors, rateLimit
      modules/
        auth/ users/ organizers/ tournaments/ registrations/
        teams/ auction/ fixtures/ results/ standings/
        notifications/ reports/ audit/
        importers/          CSV adapter(s)                              [added]
        accounts/           recovery + deletion requests                [added]
      sockets/              auction + live namespaces
      scripts/              seed, create-admin, indexes sync            [added]
      utils/
      app.ts  server.ts     (app.ts builds Express; server.ts listens)
    tests/{unit,integration,socket}/
    .env.example
  e2e/                      Playwright tests                            [moved from tests/]
  docs/                     architecture, API notes, ADRs, runbooks
  package.json              root scripts only (dev, lint, test, build)
  .gitignore  README.md  .editorconfig
```

Module rule: each module exposes *routes.ts*, *controller.ts*, *service.ts*, *model.ts*, *schemas.ts (Zod)*, *tests*. A module may import another module's *service*, not its controller or routes.

4. Dependency overview (phase level)

Full dependency table with milestone acceptance criteria is in Part 4. The critical path is:

```
A (decisions) -> B (repo) -> C (Atlas) -> D (backend) + E (frontend) -> F (DB design)
 -> G (auth/admin) -> H (public shell) -> I (tournaments) -> J (registrations)
 -> K (captains/teams) -> L (auction config) -> M (live auction)
 -> N/O/P (fixtures + standings) -> Q (manual fixtures) -> R (manual results)
 -> S (CSV, needs sample file) -> T (corrections) -> U (notifications) -> V (realtime hardening)
 -> W (dashboards) -> X (archives) -> Y (exports) -> Z (privacy/lifecycle) -> AA (audit UI)
 -> AB (polish) -> AC (QA hardening) -> AD (deployment)
Parallelizable: E alongside D; U can start right after J; N, O, P can run alongside M.
```

5. Risks, missing inputs and recommendations

5.1 Top risks

Risk	Impact	Mitigation
Auction concurrency bugs (double sale, purse overspend)	Critical: core trust feature	Atomic conditional updates, transactions, per-lot mutex, property/concurrency tests before any UI (Phase M).
Free-tier hosting sleeps or drops WebSockets	Auction experience breaks	Pick a host with persistent processes, or schedule auctions and warm the server; polling fallback; state recovered from DB on reconnect.
Free Atlas tier limits (storage, connections, no guaranteed backups)	Data loss / outages	mongodump to a private location on a schedule, tested restore, connection pooling limits, avoid unbounded collections.
Session cookies across two domains (client and API on different sites)	Login fails on Safari/mobile	Same-site custom domain, or serve client from the API host, or reverse-proxy /api. Decide before Phase D.
CSV format unknown	Rework of results/stats	Isolated adapter; build manual results first; BLOCK Phase S until sample exists.
Rule changes after start causing cascading recalculation	Wrong standings/brackets	Pure-function standings, impact preview (dry run), snapshots, transactions.
Scope size (24 modules) for a solo developer	Burnout / stall	Vertical slices, milestone releases (M1 to M8 in Part 4), keep every phase shippable.
No email verification	Fake accounts	Rate limits, per-IP throttles, organizer approval, suspension, duplicate checks.
Account deletion vs history	Legal/privacy ambiguity	Anonymize user (name -> "Deleted player", email scrubbed, credentials removed) while keeping ids on results/audit.
Windows tooling (native modules, line endings, path length)	Install friction	Use Node LTS, enable Git autocrlf=input or .gitattributes, avoid OneDrive-synced folders for the repo.

5.2 Missing inputs you must supply

- Sport/game and a **real sample scorecard CSV** (blocking Phase S).
- Confirmation of the auction "round" definition (issue 2 above).
- Invite-only mechanism preference (codes vs links vs per-user invites).
- Default points/tie-breaker presets you want pre-filled.
- Registration/auction field list (e.g. does a player have a role, skill level or base category that affects base price?).
- Privacy retention rules you want (e.g. how long audit logs and deleted-user shells are kept).
- Hosting budget ceiling and whether a custom domain is acceptable.
- Whether the admin support contact (for recovery) is an in-app form, an external email, or a messaging handle.

5.3 Additional recommendations (not in the Blueprint)

Addition	Why	First release?
sessions collection	Real logout, revocation after recovery/suspension	Essential
tournamentRuleRevisions collection	Rule history and impact previews without bloating tournaments	Essential
invites collection	Implements invite-only registration	Essential
auctionSessions collection	Holds auction config, round counter, countdown, state per tournament; keeps lot docs small	Essential
importJobs collection	Stores CSV preview data before confirmation	Essential (with Phase S)
resultRevisions collection	Immutable history of results/corrections	Essential
counters collection	Atomic waiting-list positions and sequence numbers	Essential
Health endpoint + uptime monitor	Catches sleeping/crashed host	Essential
Seed script + demo tournament	Fast manual testing of auctions/fixtures	Strongly recommended
Feature flag for polling fallback	Cost/perf tuning	Optional

Addition	Why	First release?
Admin impersonation (read-only view)	Support/debug	Not recommended in v1 (privacy risk)

6. The exact first task when we begin

Do **not** install or code anything first. The first task is **Phase A, Step A1: requirement sign-off**.

- Read Part 1 issue table (section 1.2) and reply with a decision for each item, especially issues 2, 4 and 5 (round definition, invite mechanism, sport).
- Confirm TypeScript, monorepo, server-side sessions, and the extra collections in section 5.3.
- Send one real scorecard CSV (or tell me it does not exist yet, and we will defer Phase S).
- Then we start Phase B, Step B1: verify Node.js LTS, npm, Git and VS Code versions on your Windows 11 laptop, which is a 10-minute task.

I will not begin Phase B until you ask me to.